

Module – 3

8051 Stack, I/O Port Interfacing and Programming

8051 Stack

- The stack is a section of RAM used by the CPU to store information temporarily.
 - This information could be data or an address.
- The register used to access the stack is called the SP (stack pointer) register.
 - The stack pointer in the 8051 is only 8 bit wide, which means that it can take value of 00 to FFH.
 - When the 8051 is powered up, the SP register contains value 07.
 - RAM location 08 is the first location begin used for the stack by the 8051.
- The storing of a CPU register in the stack is called a **PUSH**
 - SP is pointing to the last used location of the stack
 - As we push data onto the stack, the SP is incremented by one
 - This is different from many microprocessors
- Loading the contents of the stack back into a CPU register is called a POP
 - With every pop, the top byte of the stack is copied to the register specified by the instruction and the stack pointer is decremented once

Example 3-1

Show the stack and stack pointer from the following. Assume the default stack area.

```
MOV R6, #25H
MOV R1, #12H
MOV R4, #0F3H
```

PUSH 6

PUSH 1

PUSH 4

Solution:

	After PUSH 6	After PUSH 1	After PUSH 4
0B			
0A			F3
09		12	12
08	25	25	25
Start SP = 07	SP = 08	SP = 09	SP = 0A

Example 3-2

Examining the stack, show the contents of the register and SP after execution of the following instructions. All value are in hex.

POP 3 ; POP stack into R3

POP 5 ; POP stack into R5

POP 2 ; POP stack into R2

Solution:

	After POP 3	After POP 5	After POP 2
0B 54	0B	0B	0B
0A F9	0A F9	0A	0A
09 76	09 76	09 76	09
08 6C	08 6C	08 6C	08 6C
Start SP = 0B	SP = 0A	SP = 09	SP = 08

- The CPU also uses the stack to *save the address* of the instruction just below the CALL instruction
 - This is how the CPU knows *where to resume when it returns* from the called subroutine.
- The reason of incrementing SP after push is
 - Make sure that the stack is growing toward RAM location 7FH, from lower to upper addresses.
- If the stack pointer were decremented after push
 - We would be using RAM locations 7, 6, 5, etc. which belong to R7 to R0 of bank 0, the default register bank.
- When 8051 is powered up, register bank 1 and the stack are using the same memory space
 - We can reallocate another section of RAM to the stack.

Example 3-3

Examining the stack, show the contents of the register and SP after execution of the following instructions. All value are in hex.

MOV SP, #5FH ; make RAM location 60H, first stack location

MOV R2, #25H

MOV R1, #12H

MOV R4, #0F3H

PUSH 2

PUSH 1

PUSH 4

Solution:

	After PUSH 2	After PUSH 1	After PUSH 4
63	63	63	63
62	62	62	62 F3
61	61	61 12	61 12
60	60 25	60 25	60 25
Start SP = 5F	SP = 60	SP = 61	SP = 62

Stack and Subroutine instructions

CALLs and Subroutines

- Call instruction is used to call **subroutine**
 - Subroutines are often used to *perform tasks that need to be performed frequently*
 - This makes a program more *structured* in addition to *saving memory space*
- **LCALL** (long call)
 - 3-byte instruction
 - First byte is the opcode
 - Second and third bytes are used for address of target subroutine
 - Subroutine is located anywhere within 64K byte address space
- **ACALL** (absolute call)
 - 2-byte instruction
 - 11 bits are used for address within 2K-byte range
- When a subroutine is called, *control is transferred* to that subroutine, the processor
 - **Saves** on the stack the address of the instruction immediately below the LCALL
 - Begins to fetch instructions from the **new location**
- After finishing execution of the subroutine
 - The instruction **RET** **transfers control back** to the caller
 - Every subroutine **needs** RET as the last instruction

CALL Instructions- LCALL

1. A call opcode occurs in the program software, or an interrupt is generated in the hardware circuitry.
2. The return address of the next instruction after the call instruction or interrupt is found in the program counter.
3. The return address bytes are pushed on the stack, *low byte first*.
4. The stack pointer is incremented for each push on the stack.
5. The subroutine address is placed in the program counter.
6. The subroutine is executed.
7. A RET (return) opcode is encountered at the end of the subroutine.

Example 3-4

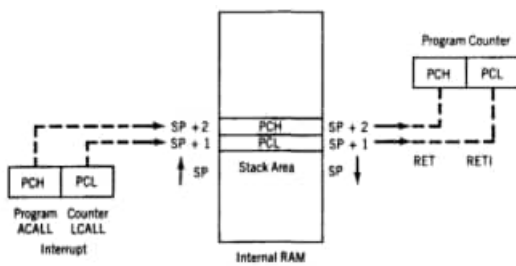
```

ORG 0
BACK: MOV A, #55H      ; load A with 55H
      MOV P1, A        ; send 55H to port 1
      LCALL DELAY      ; time delay
      MOV A, #0AAH     ; load A with AA (in hex)
      MOV P1, A        ; send AAH to port 1
      LCALL DELAY
      SJMP BACK        ; keep doing this indefinitely

;----- this is delay subroutine -----
      ORG 300H         ; put DELAY at address 300H
DELAY: MOV R5, #0FFH   ; R5=255 (FF in hex), counter
AGAIN: DJNZ R5, AGAIN  ; stay here until R5 become 0
      RET              ; return to caller (when R5 =0)
      END              ; end of asm file
  
```

Upon executing "LCALL DELAY", the address of instruction below it, "MOV A,#0AAH" is pushed onto stack, and the 8051 starts to execute at 300H.

When R5 becomes 0, control falls to the RET which pops the address from the stack into the PC and resumes executing the instructions



```

001 0000          ORG 0
002 0000 7455  BACK: MOV A,#55H ;load A with 55H
003 0002 F590      MOV P1,A ;send 55H to p1
004 0004 120300    LCALL DELAY ;time delay
005 0007 74AA      MOV A,#0AAH ;load A with AAH
006 0009 F590      MOV P1,A ;send AAH to p1
007 000B 120300    LCALL DELAY
008 000E 80F0      SJMP BACK ;keep doing this
009 0010
010 0010 ;-----this is the delay subroutine-----
011 0300          ORG 300H
012 0300  DELAY:
013 0300 7DFF      MOV R5,#0FFH ;R5=255
014 0302 DDFF  AGAIN: DJNZ R5,AGAIN ;stay here
015 0304 22        RET ;return to caller
016 0305          END ;end of asm file

```

Stack frame after the first LCALL

0A	
09	00
08	07

SP = 09

Low byte goes first
and high byte is last

CALL Instructions- ACALL

- The only difference between ACALL and LCALL is
 - The target address for LCALL can be anywhere within the 64K byte address
 - The target address of ACALL must be within a 2K-byte range
- The use of ACALL instead of LCALL can save a number of bytes of program ROM space.

Interrupts and Returns

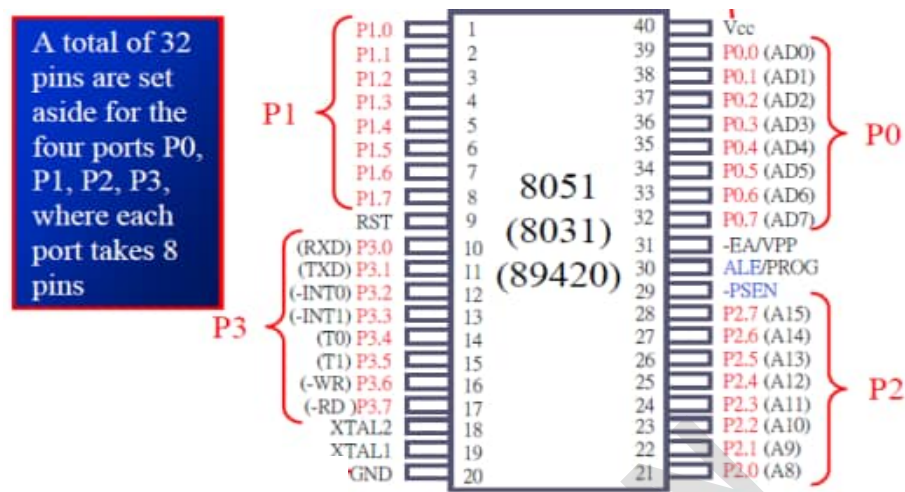
- An *interrupt* is a hardware-generated **call**.
- Just as a call opcode can be located within a program to automatically access a subroutine, certain pins on the 8051 can **cause a call** when external electrical signals on them **go to a low state**.
- Internal operations of the **timers** and the **serial port** can also cause an interrupt call to take place.
- The subroutines called by an interrupt are located at fixed hardware addresses and are called as **Interrupt Service Routine (ISR)**.
- When an interrupt call takes place,
- *Hardware interrupt disable flip-flops* are set.
- So that it prevents another interrupt of the same priority level from taking place.
- Disabled Hardware interrupts can be enabled back by executing an *interrupt return* instruction. (Generally written at the end of the interrupt subroutine.)

INTERRUPT	ADDRESS (HEX) CALLED
IEO	0003
TFO	0008
IE1	0013
TF1	0018
SERIAL	0023

RETI

- Pops two bytes from the stack into the program counter (PC) and reset the interrupt enable Flip-Flops.
- The only difference between the RET and RETI instructions is the enabling of the interrupt logic when RETI is used.
- RET is used at the ends of subroutines called by an opcode. RETI is used by subroutines called by an interrupt

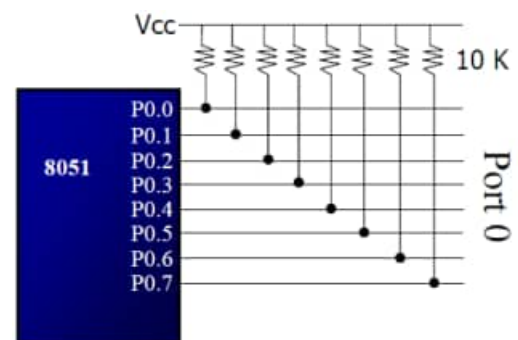
I/O Port Interfacing and Programming



- The four 8-bit I/O ports P0, P1, P2 and P3 each uses 8 pins
- All the ports upon RESET are configured as input, ready to be used as input ports
 - When the first 0 is written to a port, it becomes an output
 - To reconfigure it as an input, a 1 must be sent to the port
 - To use any of these ports as an input port, it must be programmed

Port 0

- Port 0 can be used for input or output, each pin must be connected externally to a 10K ohm pull-up resistor
 - This is due to the fact that P0 is an open drain, unlike P1, P2, and P3
 - Open drain is a term used for MOS chips in the same way that open collector is used for TTL chips



Example 3-5

The following code will continuously send out to port 0 the alternating value 55H and AAH

```
BACK:    MOV     A, #55H
          MOV     P0, A
          ACALL   DELAY
          MOV     A, #0AAH
          MOV     P0, A
          ACALL   DELAY
          SJMP    BACK
```

- In order to make port 0 an input, the port must be programmed by writing 1 to all the bits

Example 3-6

Port 0 is configured first as an input port by writing 1s to it, and then data is received from that port and sent to P1

```

MOV    A, #0FFH    ; A=FF hex
MOV    P0, A        ; make P0 an i/p port by writing it all 1s
BACK:  MOV    A, P0    ; get data from P0
        MOV    P1, A    ; send it to port 1
        SJMP   BACK    ; keep doing it

```

Port 1

- Port 1 can be used as input or output
- In contrast to port 0, this port does not need any pull-up resistors since it already has pull-up resistors internally
 - Upon reset, port 1 is configured as an input port
 - To make port 1 an input port, it must be programmed as such by writing 1 to all its bits

Example 3-7

The following code will continuously send out to port 1 the alternating value 55H and AAH

```

MOV    A, #55H
BACK:  MOV    P1, A
        ACALL DELAY
        CPL   A
        SJMP  BACK

```

A= 01010101(55h) =>Complementing gives 10101010(AAh)

Example 3-8

Port 1 is configured first as an input port by writing 1s to it, then data is received from that port and saved in R7 and R5

```

MOV    A, #0FFH    ; A=FF hex
MOV    P1, A        ; make P1 an input port by writing it all 1s
MOV    A, P1        ; get data from P1
MOV    R7, A        ; save it to in reg R7
ACALL  DELAY        ; wait
MOV    A, P1        ; another data from P1
MOV    R5, A        ; save it to in reg R5

```


Port 2 and Port 3

- Port 2 can be used as input or output
 - Just like P1, port 2 does not need any pull-up resistors since it already has pull-up resistors internally
 - Upon reset, port 2 is configured as an input port
- To make port 2 an input port, it must be programmed as such by writing 1 to all its bits.
- Port 3 can be used as input or output
 - Port 3 does not need any pull-up resistors
 - Port 3 is configured as an input port upon reset.

I/O Programming- Different ways of Accessing Entire 8 Bits

The entire 8 bits of Port 1 are accessed

```
BACK:    MOV    A, #55H
          MOV    P1, A
          ACALL  DELAY
          MOV    A, #0AAH
          MOV    P1, A
          ACALL  DELAY
          SJMP   BACK
```

Rewrite the code in a more efficient manner by accessing the port directly without going through the accumulator

```
BACK:    MOV    P1, #55H
          ACALL  DELAY
          MOV    P1, #0AAH
          ACALL  DELAY
          SJMP   BACK
```

Another way of doing the same thing

```
BACK:    MOV    A, #55H
          MOV    P1, A
          ACALL  DELAY
          CPL    A
          SJMP   BACK
```

Example 3-9

Write a program to toggle all the bits of port1 by sending to it the values 55h and AAh continuously. Put a time delay in between each issuing of data to port1.

```
ORG 0
BACK:    MOV    A, #55H    ; Load A with 55h
          MOV    P1, A     ; Send 55h to port1
          ACALL  DELAY     ; Call the subroutine DELAY
          MOV    A, #0AAH  ; Load A with AAh
          MOV    P1, A     ; Send AAh to port1
          ACALL  DELAY     ; Call the subroutine DELAY
          SJMP   BACK      ; Keep doing this indefinitely

----- Delay Subroutine-----
ORG 300H
DELAY:    MOV    R5, #0FFH ; R5 works as counter with 256 counts
AGAIN:    DJNZ   R5, AGAIN ; Stay here until R5 becomes 0
          RET                     ; Return to Called program
          END
```

Example 3-10

Rewrite the previous program **efficiently**.

```

                ORG 0
                MOV A, #55H      ; Load A with 55h
BACK:           MOV P1, A        ; Send 55h to port1
                ACALL DELAY      ; Call the subroutine DELAY
                CPL A            ; Complements A, so that A becomes AAh
                SJMP BACK        ; Keep doing this indefinitely
----- Delay Subroutine-----
                ORG 300H
DELAY:          MOV R5, #0FFH    ; R5 works as counter with 256 counts
AGAIN:         DJNZ R5, AGAIN    ; Stay here until R5 becomes 0
                RET              ; Return to Called program
                END

```

Example 3-11

Write a program to toggle the bits of port1 delay which depends on the value of a number in R0.

```

                ORG 0
BACK:           MOV A, #0H
                MOV P1, A        ; Send 0h to port P1
                MOV R0, #30H     ; Required count value for generating delay
                ACALL DELAY      ; Call the subroutine DELAY
                CPL A            ; Complement A to toggle
                MOV P1, A        ; Send AAh to port1
                MOV R0, #0FFH    ; required count value for generating delay
                ACALL DELAY      ; Call the subroutine DELAY
                SJMP BACK        ; Keep doing this indefinitely
----- Delay Subroutine-----
                ORG 300H
DELAY:          NOP              ; Do nothing
AGAIN:         DJNZ R0, AGAIN    ; Stay here until R0 becomes 0
                RET              ; Return to the Called program
                END

```

Example 3-12

Write a program to perform the following:

- (a) Keep monitoring the P1.2 bit until it becomes high
- (b) When P1.2 becomes high, write value 45H to port 0
- (c) Send a high-to-low (H-to-L) pulse to P2.3

```

                SETB P1.2        ; make P1.2 an input
                MOV A, #45H      ; A=45H
AGAIN:         JNB P1.2, AGAIN   ; get out when P1.2=1
                MOV P0, A        ; issue A to P0
                SETB P2.3        ; make P2.3 high
                CLR P2.3         ; make P2.3 low for H-to-L

```


Example 3-13

Assume that bit P2.3 is an input and represents the condition of an oven. If it goes high, it means that the oven is hot. Monitor the bit continuously. Whenever it goes high, send a high-to-low pulse to port P1.5 to turn on a buzzer.

```
HERE:      JNB    P2.3, HERE    ; keep monitoring for high
           SETB   P1.5          ; set bit P1.5=1
           CLR    P1.5          ; make high-to-low
           SJMP   HERE          ; keep repeating
```

Example 3-14

A switch is connected to pin P1.7. Write a program to check the status of SW and perform the following:

- (a) If SW=0, send letter 'N' to P2
- (b) If SW=1, send letter 'Y' to P2

```
           SETB   P1.7          ; make P1.7 an input
AGAIN:     JB     P1.7, OVER     ; jump if P1.7=1
           MOV    P2, #'N'       ; SW=0, issue 'N' to P2
           SJMP   AGAIN          ; keep monitoring
OVER:      MOV    P2, #'Y'       ; SW=1, issue 'Y' to P2
           SJMP   AGAIN          ; keep monitoring
```

Example 3-15

A switch is connected to pin P1.0 and an LED to pin P2.7. Write a program to get the status of switch and send it the LED.

```
           SETB   P1.0          ; make P1.0 an input
AGAIN:     MOV    C, P1.0        ; Read the switch status in to CF
           MOV    P2.7, C        ; Send the switch status on to LED
           SJMP   AGAIN          ; keep monitoring
```

ಎಣ್ಣೆಯಿಲ್ಲದೆಯೂ
ಒಮ್ಮೊಮ್ಮೆ ದೀಪಗಕಾರುತ್ತವೆ.
ಪ್ರತಿ ಬಾರಿಯೂ ಗಾಳಿಯ ತಪ್ಪು ಖರಲಾರದಲ್ಲವೆ?
ಕಲಿಕೆಯ ದೀಪದಲ್ಲ ಉತ್ಸಾಹದ ಎಣ್ಣೆ ಬತ್ತದಿರಲಿ

VTUSYNC.IN